DATA BASES – PROF. D.M. BRAGA – A. A. 2021/22

ALMOST-THE-VERY-BEGINNING-OF-2022 … ACTUALLY … WELL-YEAH-ERRRR-ALMOST. REALLY.

DURATION: 1h 30m

A. SQL (24 points)

PoliSaplings is an environmentalist no-profit organization that, every December, distributes conifer saplings for indoor decoration to supporters who commit themselves to planting them in a suitable habitat right after the *winter* break. Assignments are done in exchange for a deposit that may be fully reimbursed during the *spring* break, on evidence that the sapling is properly installed and in good shape. The outcome of a status check can be "KO", "OK", or "Recheck", to state that a new check will be performed later, overwriting (with an update) the date and the outcome of the previous check, until a final positive or negative outcome is recorded.



SAPLING (<u>TREEID</u>, CULTIVAR, HEIGHT, DIAMETER) // CULTIVAR can be *Fraser Fir*, *Blue Sprouce*, *White Pine, …*
SUPPORTER (<u>SUPPID</u>, NAME, PHONE, ADDRESS, BIRTHDATE)
ASSIGNMENT (<u>TREEID</u>, SUPPID, DATE, DEPOSIT)
STATUSCHECK (<u>TREEID</u>, DATE, HEIGHT, DIAMETER, OUTCOME) // OUTCOME can be *OK*, *KO*, *or Recheck*

1. Describe as compactly and precisely as possible what is extracted by these queries [**5 points**]

a)
```
select S.*, C.Height – S.Height
from Sapling S join StatusCheck C on S.TreeId=C.TreeId
where C.Height – S.Height >= ALL ( select C1.Height – S1.Height
                                   from Sapling S1 join StatusCheck C1 on S1.TreeId=C1.TreeId
                                   where S1.Cultivar = S.Cultivar )
```

b)
```
select SuppId
from Assignment
where SuppId not in ( select A1.SuppId
                      from ( Assignment A1 natural join Sapling S1 ) join ( Assignment A2 natural join Sapling S2 )
                           on A1.SuppId = A2.SuppId
                      where A1.Date < A2.Date and S1.Height >= S2.Height )
group by SuppId having count(*) >= 5
```

2. Express in SQL[**15 points**] :

   a. The average height and diameter of the saplings of each cultivar [**4 p.**];

   b. For each year, the name of the supporter who was assigned the *tallest* sapling (*function* `year(Date)` *extracts the year from a date*) [**5 p**.];

   c. The supporters who, despite their 10 or more assignments, always failed to have their deposits refunded (as the status check of their trees never yielded a positive outcome so far) [**6 p**.].

SAPLING (TREEID, CULTIVAR, HEIGHT, DIAMETER) // CULTIVAR can be *Fraser Fir*, *Blue Sprouce*, *White Pine, …*

SUPPORTER (SUPPID, NAME, PHONE, ADDRESS, BIRTHDATE)

ASSIGNMENT (TREEID, SUPPID, DATE, DEPOSIT)

STATUSCHECK (TREEID, DATE, HEIGHT, DIAMETER, OUTCOME) // OUTCOME can be *OK*, *KO*, *or Recheck*

3. If, on positive status checks, deposits are immediately refunded, the following **materialized** view shows the financial balance of each supporter (the amount of money paid and not refunded).

```
create view GlobalBalance( Sid, Balance ) as
select SuppId, coalesce( sum( Deposit ), 0 ) – ( select coalesce( sum( Deposit ), 0 )
                                                 from Assignment natural join StatusCheck
                                                 where Outcome="OK" and SuppId = A.SuppId )
from Supporter natural left join Assignment A
group by SuppId
```

Assume that there is a referential integrity constraint Assignment(SuppId) → Supporter(SuppId). Formulate a set of triggers that incrementally maintain the materialization in response to insertions on table ASSIGNMENT and to insertions and updates on table STATUSCHECK. Please note that, due to the referential integrity, "zero tuples" in the view (tuples <id,0> for new supporters who haven't yet had their first assignment) are already in place when saplings are assigned or checked. They are added by some trigger reacting to insertions on table SUPPORTER, not reqested in the solution for brevity [4 points]

B. Conceptual Design (8 points)

Design an E-R model for the database described in the following scenario:

An on-demand streaming service allows subscribers to watch a variety of TV shows (series, movies, documentaries…). Subscribers can register by their email and a password, set a desired video quality, and restrict the service to wifi connections. For each subscriber, the DB records name, date of the subscription, expiration date, and the chosen method of payment. A subscriber can create multiple user profiles, each characterized by an image, a name, and an indication if it is for adults or children. TV series are categorized into seasons (each characterized by a number) and composed of episodes, also numbered. All shows are characterized by a title, an image, year of production, date of publication on the streaming service, duration, and a link to the actual video content. The DB tracks what has been viewed by each profile, and if a video was only partially viewed it also records how many minutes/seconds were viewed.

- *The ideal schema is complete, legible, minimal, self-explanatory, and of course also correct.*
- *Remember to specify **identifiers** for all entities and **cardinalities** for all associations.*